How the work on RWCrew is tested

Test scenarios and test method used while building RWCrew · status of 8 November 2026 · branch `develop`

In short. Every change is checked at several levels. The main safety net is an automated backend test suite of **1026 tests** that calls the real API on a throw-away database. Frontend changes are checked by the TypeScript compiler and the linter, and some of them in a real browser against localhost. Larger data loads are rehearsed with the mass test tool. Every check, and every check that was *not* done, is written in the **Verification** line of `docs/work-history.md`.

1. The test levels

1 · Backend tests	**pytest**, 1026 tests in 102 files. They run on every change, always on the module that changed and the **full suite** before work is called done or committed.
2 · Frontend checks	`tsc --noEmit` (types) and **eslint** on every frontend change. There is no automated frontend test suite yet.
3 · Browser / app	For screens: open them on localhost (Edge, through playwright-core scripts), check light/dark mode and phone width. For PDFs: generate them and look at the pages.
4 · Mass test	Last year's real lists loaded through the normal API (localhost → test → production), with an Excel issue report of every row that fails.
5 · Real data	Migrations run on the local Postgres database (e.g. "all 232 kars got their code"), and you test on the TEST server before production.

2. How a backend test works

2.1 A fresh database for every test

Each test gets its own **empty in-memory SQLite database** (`tests/conftest.py`). All tables are made from the SQLAlchemy models, the test builds the small world it needs, and the database is gone when the test ends. So tests never depend on each other, never need Docker, and **never touch your local or server data**.

- **Foreign keys switched on** (`PRAGMA foreign_keys=ON`), so deleting a user cascades the same way as on Postgres.
- **Files go to temporary folders**: e.g. wristband `.json` files and TagScan CSV folders are redirected per test.
- **No outgoing mail**: without a Resend key, `send_email()` only logs; mail tests capture what *would* be posted and check recipients, CC and text.
- **No background jobs**: the automatic TagScan scan only starts in the app's lifespan, which tests don't run.

2.2 Through the real API, as a real user

Tests use FastAPI's `TestClient`: they send real HTTP requests to the real routers, with real login cookies, real rights checks and the real services. Only the database is swapped. A typical module has a `conftest.py` that builds a small "world", for example StockMaster's:

```
an open and a closed season, one warehouse, three products (normal, consumable with minimum stock,
blocked),
two kars of one team, a festival, the seeded reasons, and a "Magazijn" role with every right
+ helpers: make_user(...), book(...), stock_of(...)
```

A test then describes one rule in its name and checks the answer *and* what was stored:

```
def test_stock_can_never_go_negative(as_warehouse, world):
    _ten_laders_spread(as_warehouse, world)            # 5 free, 3 in kar K043, ...
    response = book(as_warehouse, world, "kar_unload", [(world.lader.id, 4)], kar_id=world.k043.id)

    assert response.status_code == 400
    assert response.json()["detail"] == "insufficient_stock|Walkie-lader|3|K043"
    # Nothing of the refused booking was kept.
    assert stock_of(as_warehouse, world.lader.id)["free"] == 5
```

2.3 What every test tends to check

Kind of check	Example
The happy path	A hand-out at the Atelier desk books the pieces from *available* to *handed out* and sends one mail.
The refusal and its exact error code	`reservation_full|team|holding|reserved`, `season_closed`, `bracelet_differs|number|name` – the frontend translates these codes, so the test pins them.
Nothing half-saved after a refusal	A blocked box in Massa-inname books **nothing**; a refused stock booking leaves every balance as it was.
Rights	No module access → 403; view but not edit → 403 on save; one screen's right doesn't open another screen.
Who did it	The booked-by user is always the logged-in user, never a value sent in the request.
Documents	Every PDF renders (also with many rows and non-Latin text); Excel exports contain the shown lines, formula-like values stay text.

3. The test scenarios per area

Area	Tests	Main scenarios
Login & access (landing, core)	71	Correct/wrong password, unknown e-mail, disabled account; logout; refresh-token rotation and **reuse detection**; the 6 h session is never extended by a refresh; Bearer login for the Pi; login history (web / phone / Pi, device type). Super admin grants and revokes module access but does **not** get it automatically; module admin only in their own module; a user can't delete themselves. Belgian time (summer UTC+2, winter UTC+1). Changelog read from GitHub with cache and fallback.
Custom roles (every module with roles)	in each module	The same set per module: screens are synced (twice is safe), no role → no access, view without edit can't change, duplicate role name refused, a role still assigned can't be deleted, the permission matrix is replaced as a whole, `me/permissions` shows exactly what the role allows, super admin bypasses.
Scans / TagScan (module-1)	201	CSV intake from the Pi (API key, duplicates, same name with other content), scanning and EPC matching, Assignment processing per file with a mandatory product, cancel with reason, Book-In lines left for StockMaster, tag ↔ kar link, file browser with path-traversal refused, PDFs, auto-scan settings, forwarding by `Mode` TEST/PROD.
Wristband scans (shared)	14	Webhook token (no token = everything refused), every message logged, scanner auto-registered, team by name or alias, a failed file write never loses the scan, refused senders leave no file, which counter sees which scan (reservation at any festival of the season), test scans.
KarTracker (module-2)	158	Kar CRUD and XLSX import, Plan een kar overview, Leveringsvolgorde (tour 1..n, others emptied, inactive untouched), initial delivery planning and kar moves between festivals incl. PDFs with map, special tours, Karcode formula (whitelist parser, never `eval`, error codes) and the stored kar code on Karblad/Ploegfiche.
Interventies (module-3)	43	Requests from staff screen, public QR form and phone, the new-request mail with PDF to the mailing list, dashboard counts, validation of addresses.
StockMaster (module-4)	102	Free stock and kar stock, **never negative**, kar-to-kar moves, book-out with reason, blocked product, kar departure/return and follow-up deliveries, counts book only the differences, undo once, closed season locked, minimum stock, kar needs → progress and "Te bestellen", delivery entitlement (flow diagram paths), manual requests, Inboeken from TagScan, voice counts, product master data and XLSX, every PDF.

Atelier (module-5)	91	Master data (sizes/fits, articles, size shares), reservations per festival and copy from last year, preparation totals, stock (receipt, wash, write-off, count, undo), counter desk (pick-up rules, 1 piece per article, **reservation limit** and override only with edit, partial/full return, cancel, other wristband needs confirmation, write-off from Openstaand), mails and templates, RFID per piece, Massa-inname, wash statistics, back-dating (super admin only, never mails).
Walkies (module-6), module-7, module-10	20	Walkies: roles and test scans (the module is still a shell). Mobile App: install info needs login, empty without a public URL, points at `/m`.
Altsien Select (module-8)	90	Team scope (a Kernlid only sees own teams, others 404), season lock (403 unless "all teams" edit), every wizard step and its completion, copy from last season (adds/updates, never removes), products step through StockMaster's service, ploegfiche screen/PDF and mail with CC, removed walkie step really gone.
MasterData (module-9)	145	Seasons, festivals, teams (unknown ids → 404), team aliases, festivals per ploeg grid (closed year 403, proposal from last year), team responsibles, XLSX import/export.
Help manuals, mass test	10	Manual chapters exist in nl and en, screen keys exist, images exist, chapters follow the screens the user may view, PDF renders whole and per topic. Mass test: converter groups rows and logs unreadable ones, a full run loads everything once and **never twice**, a run without rights stops before writing.

Counts = number of test functions in that folder; some tests run several times with different parameters, so pytest reports a few more.

4. The routine for one change

1. **Read first**: the existing code, its tests and the rules in `CLAUDE.md` (e.g. "only `stock_service.py` changes stock").
2. **Write or extend the tests together with the code**: one test per rule you asked for, named in plain words, plus the refusals and rights around it.
3. **Run the module's tests**, then the **full backend suite** – a change in one module can break another (e.g. Altsien Select uses StockMaster's kar needs).
4. **Frontend**: `tsc --noEmit` and eslint must be clean.
5. **Where it matters, look at it**: open the screen on localhost, generate the PDF, render a Pi screen off-screen with fake data.
6. **Write it down**: the Verification line in `docs/work-history.md` says what passed and what wasn't checked; the changelog line goes to `done`.

5. Protecting your own data

> **Lesson learned (24 Oct 2026):** a browser test once "reset" the delivery tour after itself and wiped your saved terrein positions. Since then: before a check writes to the local database, the current values are read first and put back exactly in a `finally`; read-only checks are preferred; and the run says up front which data it will touch. For bigger demos (e.g. the Pi screenshot demo) the local database is **dumped with `pg_dump` before** and **restored after**. The pytest suite itself can't touch real data: it only ever uses its in-memory database and temp folders.

The mass test also has guards: it refuses to write reports inside the repo (they hold personal data), keeps a ledger so a second run loads nothing twice, and on production needs `--confirm-production`, a dry run the same day and a typed confirmation that a database dump was made.

6. What this does not cover (honestly)

Gap	What it means / how it's handled
No automated frontend tests	Buttons, dialogs and filters are only checked by types, lint and – not always – a manual look in the browser. Work-history entries say "not yet checked in the browser" when that's the case.
SQLite instead of Postgres	SQLite ignores `SELECT … FOR UPDATE`, so two people booking the same stock at the same second is not really tested. Postgres-only behaviour shows up only locally or on TEST.
Migrations	Tests build tables from the models, not from Alembic. Migrations are checked by running `alembic upgrade head` on the local Postgres and looking at the result.
Outside systems	Resend, GitHub (changelog), the wristband company and the Cloudflare tunnel are replaced by fakes in tests. The first real mail on TEST failed once because a setting was missing in `docker-compose.prod.yml`.
Hardware and phones	The RFID reader, the Pi's buttons and the iPhone's speech recognition can't be tested automatically; they need a real device (e.g. voice count on iOS is on hold).

7. How to run the tests yourself

Result on 8 November 2026: full backend suite **1026 passed**, 0 failed, in 6 min 28 s (3 harmless warnings about a schema class whose name starts with "Test").

```
cd backend
.venv/Scripts/python -m pytest                                  # everything (1026 tests, ~6.5 min)
.venv/Scripts/python -m pytest tests/modules/module_5           # one module
.venv/Scripts/python -m pytest -k reservation_limit              # tests whose name matches
.venv/Scripts/python -m pytest -x -q                            # stop at the first failure, short output

cd frontend
npx tsc --noEmit                                                # type check
npx eslint src                                                  # lint
```